

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ ІМЕНІ МИХАЙЛА
ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ
ІНЖЕНЕРІЇ ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

Освітній компонент
«АЛГОРИТМИ І СТРУКТУРА ДАНИХ»

ЗВІТ
З ПРАКТИЧНОЇ РОБОТИ № 4

виконала:
студентка групи КН-25-1
Бойко Д.О.
Перевірив:
Старший викладач кафедри АІС
Сидоренко В.М.

Кременчук 2026

Практична робота № 4

Тема. Алгоритми пошуку та їх складність

Мета: опанувати основні алгоритми сортування та навчитись методам аналізу їх асимптотичної складності.

Постановка завдання:

Виконати індивідуальне завдання. Завдання полягає у розв'язку всіх задач, наведених нижче. Тобто студенти виконують єдиний варіант для всіх.

Завдання 1

Оцінити асимптотичну складність алгоритму лінійного пошуку у O -нотації в найгіршому і в найкращому випадку. Як можна покращити алгоритм лінійного пошуку?

Розв'язання

Лінійний пошук послідовно перевіряє кожен елемент масиву до знаходження потрібного значення.

Найгірший випадок

Шуканий елемент знаходиться в кінці масиву або відсутній.

Кількість операцій пропорційна кількості елементів:

$$O(n)$$

Найкращий випадок

Елемент знаходиться на першій позиції.

$$O(1)$$

Способи покращення

Для покращення ефективності можна:

- впорядкувати масив;
- використовувати бінарний пошук;
- застосовувати хеш-таблиці.

Висновок

Лінійний пошук є простим у реалізації, але малоефективним для великих масивів, оскільки має лінійну складність.

Завдання 2

Оцінити асимптотичну складність алгоритму бінарного пошуку у O -нотації в найгіршому і в найкращому випадку.

Розв'язання

Найгірший випадок

Кількість перевірок:

$$O(\log n)$$

Найкращий випадок

Шуканий елемент одразу знаходиться в центрі масиву.

$$O(1)$$

Висновок

Бінарний пошук значно швидший за лінійний для великих відсортованих масивів.

Завдання 3

Побудувати алгоритм тернарного пошуку і оцінити його асимптотичну складність алгоритму у O -нотації в найгіршому і в найкращому випадку. Який з алгоритмів є оптимальнішим: бінарний, чи тернарний? Обґрунтувати відповідь відповідними обчисленнями.

Розв'язання

Тернарний пошук ділить масив на три частини.

Код алгоритму:

```
1 def ternary_search(arr, left, right : (_ge_), x): 3+ usages
2
3     if right >= left:
4
5         mid1 = left + (right - left) // 3
6         mid2 = right - (right - left) // 3
7
8         if arr[mid1] == x:
9             return mid1
10
11        if arr[mid2] == x:
12            return mid2
13
14        if x < arr[mid1]:
15            return ternary_search(arr, left, mid1 - 1, x)
16
17        elif x > arr[mid2]:
18            return ternary_search(arr, mid2 + 1, right, x)
19
20        else:
21            return ternary_search(arr, mid1 + 1, mid2 - 1, x)
22
23    return -1
```

Рисунок 1-Код програми

Найгірший випадок

$$O(\log_3 n)$$

Найкращий випадок

$$O(1)$$

Порівняння з бінарним пошуком

Бінарний пошук є ефективнішим, тому що:

- виконує менше порівнянь;
- має простішу реалізацію;
- швидше працює на практиці.

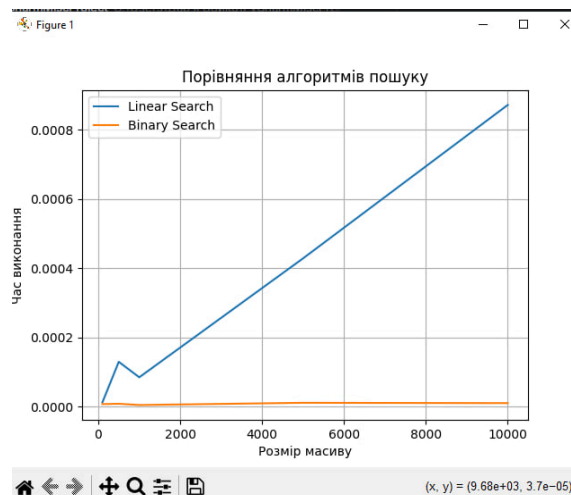
Висновок

Тернарний пошук також є швидким алгоритмом, однак бінарний пошук використовується частіше через кращу практичну ефективність.

Завдання 4

Порівняти ефективність алгоритмів лінійного, бінарного та тернарного пошуку для різних розмірів вхідного списку. Для цього провести експериментальне дослідження та побудувати графіки залежності часу виконання алгоритму від розміру вхідного списку.

Рисунок 2-Графік



Алгоритм	Складність
Лінійний	$O(n)$
Бінарний	$O(\log n)$
Тернарний	$O(\log n)$

Лінійний пошук: демонструє прямолінійне зростання часу виконання, що на практиці підтверджує його складність $O(n)$.

Бінарний та тернарний пошуки: показники обох алгоритмів знаходяться близько до осі абсцис, що характерно для логарифмічної складності $O(\log n)$.

Тернарний пошук працює дещо довше за бінарний через збільшену кількість операцій порівняння на кожному етапі.

Завдання 5

Порівняти алгоритми пошуку за їхньою здатністю працювати з відсортованими та не відсортованими списками. Провести аналіз впливу відсортованості списку на час виконання кожного алгоритму.

Лінійний пошук може працювати як з відсортованими, так і з невідсортованими списками. Алгоритм послідовно перевіряє кожен елемент масиву до знаходження потрібного значення.

Складність алгоритму:

- найкращий випадок — $O(1)$
- найгірший випадок — $O(n)$

Бінарний пошук працює тільки з відсортованими списками. Алгоритм ділить область пошуку навпіл.

Складність алгоритму:

- найкращий випадок — $O(1)$
- найгірший випадок — $O(\log n)$

Тернарний пошук також потребує відсортованого списку та ділить масив на три частини.

Складність алгоритму:

- найкращий випадок — $O(1)$
- найгірший випадок — $O(\log_3 n)$

Порівняння алгоритмів

Алгоритм	Працює з невідсортованим списком	Працює з відсортованим списком	Складність
Лінійний пошук	так	так	$O(n)$
Бінарний пошук	ні	так	$O(\log n)$
Тернарний пошук	ні	так	$O(\log_3 n)$

Лінійний пошук не залежить від впорядкованості списку, тому може застосовуватись до будь-яких даних. Проте при великих розмірах списку його ефективність зменшується через необхідність перевірки великої кількості елементів.

Бінарний та тернарний пошук працюють значно швидше, але потребують попереднього сортування списку. Якщо список не відсортований, використання цих алгоритмів є неможливим.

Для великих обсягів даних найбільш ефективним є бінарний пошук, оскільки він має логарифмічну складність та виконує меншу кількість порівнянь.

Висновок

У ході виконання завдання було проведено порівняння алгоритмів пошуку для відсортованих та невідсортованих списків. Встановлено, що лінійний пошук є універсальним, але менш ефективним для великих масивів. Бінарний та тернарний пошук забезпечують значно швидшу роботу, проте можуть використовуватись лише для відсортованих даних.

Завдання 6

Розглянути сценарії використання кожного з алгоритмів пошуку у практичних задачах і обґрунтувати вибір кожного алгоритму в конкретному випадку.

- Якщо дані не відсортовані, то використовується лінійний пошук.
- Якщо дані відсортовані та великі, то використовується бінарний пошук.
- Тернарний пошук використовується рідко, лише у спеціальних математичних задачах, майже не зустрічається у типовому програмному забезпеченні.

Контрольні питання

1. Що таке алгоритм пошуку і чому він важливий у контексті комп'ютерних наук?

Алгоритм пошуку — це послідовність дій для знаходження потрібного елемента в структурі даних (масиві, списку, базі даних). Він важливий, тому що дозволяє швидко знаходити інформацію у великих обсягах даних, що є основою роботи програм, баз даних і пошукових систем.

2. Які основні критерії оцінки ефективності алгоритмів пошуку?

Основні критерії:

- час виконання (швидкість роботи);
- кількість операцій порівняння;
- використання пам'яті;
- асимптотична складність

3. Що таке лінійний пошук, і як він працює?

Лінійний пошук — це алгоритм, який послідовно перевіряє кожен елемент списку, поки не знайде потрібний або не перегляне всі елементи.

Складність: $O(n)$.

4. Які умови повинні бути виконані для успішного застосування бінарного пошуку?

- масив повинен бути відсортований;
- дані мають бути доступні за індексом;
- елементи повинні бути порівнюваними.

5. Які переваги та недоліки використання бінарного пошуку порівняно з іншими алгоритмами пошуку?

Переваги:

- дуже швидкий ($O(\log n)$) ;
- значно менше порівнянь, ніж у лінійному пошуку.

Недоліки:

- працює тільки з відсортованими даними;
- складніший у реалізації.

6. Що таке тернарний пошук, і в чому його відмінність від бінарного пошуку?

Тернарний пошук — це алгоритм, який ділить масив на три частини і шукає елемент у відповідному інтервалі.

Відмінність:

- бінарний ділить на 2 частини;
- тернарний — на 3 частини;
- бінарний зазвичай ефективніший на практиці, хоча обидва мають логарифмічну складність.